

Redux – FAANG / MAANG Interview Notes

1. One-line FAANG Answer (Memorize)

Why Redux instead of local state?

Redux is needed because the app has large, shared, long-lived business state that must stay consistent across many components. Local state would cause prop drilling, duplication, and state mismatch.

2. Why Redux is Required (Core Reasons)

Business State, not UI State

The app manages customer data, items, payments, totals, delivery rules, and signatures. This is transactional domain state, which Redux is designed for.

Shared Across Many Components

State is used by LeftPanel, PackageMaker, SignaturePad, Payment sections, and Delivery Calendar. Without Redux, state must be lifted high and passed through many props, leading to prop drilling bugs.

Cascading Calculations

One change updates subtotal, tax, total, balance, payment terms, and delivery pricing. Redux ensures a single source of truth so values never go out of sync.

Deterministic Updates

Redux updates state only through actions, making changes predictable, traceable, and debuggable.

Clean UI vs Domain Separation

UI-only state stays local (modals, collapse flags). Business-critical data stays in Redux. This is senior-level architecture.

3. Why Not Context API?

Context is good for sharing data, Redux is good for controlling complex state. Context lacks action history, predictable updates, and scalability. Redux Toolkit provides centralized logic and better debugging.

4. Interview Golden Sentence

I used Redux because the app has tightly coupled business state where a change in one part must consistently affect many other parts. Redux ensures a single source of truth, predictable updates, and better scalability.

5. Improvements (Bonus Points)

Issue: Too much logic in components.

Fix: Move calculations to Redux selectors or reducers.

Issue: Business flags like addressVerified stored locally.

Fix: Move them to Redux (e.g., state.billToVerified).

Issue: Too many useEffect recalculations.

Fix: Centralize calculations in Redux and keep React focused on UI.

6. Final Verdict

Redux is not optional here. It is required to keep the app correct, predictable, and scalable.

7. Spoken Interview Practice (English)

Practice explaining Redux decisions clearly and confidently, focusing on business state, scalability, and predictability.

8. Mock FAANG Follow-up Questions

- Why Redux over Context?
- How do you avoid overusing Redux?
- What state should never go into Redux?

9. Redux vs Zustand / Jotai (Quick Comparison)

Redux is best for large, enterprise apps with complex workflows. Zustand/Jotai are lighter and better for small to medium apps with simpler state needs.

10. Refactoring to MAANG Style

Move business logic out of components into Redux selectors, reducers, or middleware. Keep components lean and declarative.